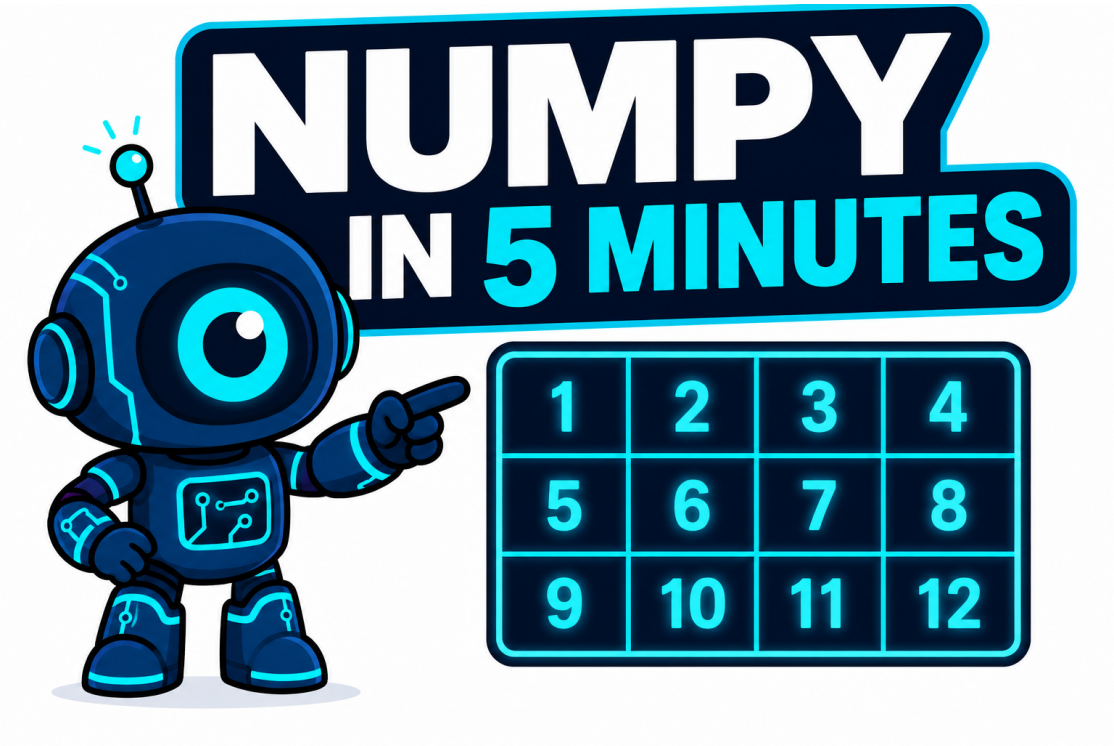
AI MECH LAB

AI Course — Beginner Track



Module 2, Topic 2.2

NumPy Basics

Lesson

You now know Python's basic building blocks. This lesson introduces the first specialized tool you'll use throughout this course: NumPy. If you remember one thing about why NumPy exists, remember this — regular Python is slow at handling large amounts of numerical data, and NumPy fixes that.

Why not just use a regular Python list?

You already know how to store numbers in a list:

```
readings = [42, 45, 41, 50, 47]
```

This works fine for small amounts of data. But AI and machine learning work involves numerical data at a much larger scale — thousands or millions of values, often organized into grids (rows and columns) rather than a single flat list. Python's built-in lists weren't designed for that kind of work; operations on them get slow as the data grows, and there's no built-in way to do math across an entire list at once.

NumPy introduces a new data structure called an array, which looks similar to a list on the surface but is built specifically for fast numerical computation.

```
import numpy as np

readings = np.array([42, 45, 41, 50, 47])
```



A Python list and a NumPy array look similar, but the array is built for speed.

The import numpy as np line loads the library and gives it a short nickname, np, which is the near-universal convention you'll see in essentially every piece of NumPy code you ever read.

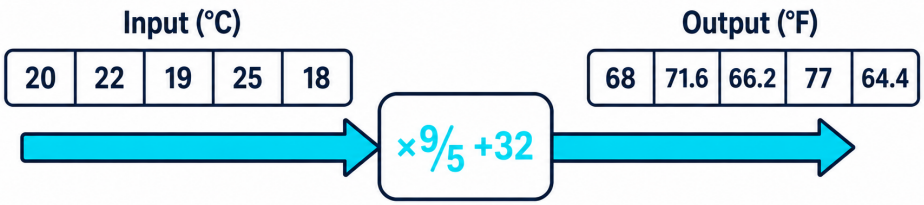
The core advantage: operating on entire arrays at once

Here's where NumPy actually earns its place in this course. Suppose you want to convert every reading in your list from Celsius to Fahrenheit. With a regular Python list, you'd need a loop:

```
readings = [20, 22, 19, 25]
fahrenheit = []
for temp in readings:
    fahrenheit.append((temp * 9/5) + 32)
```

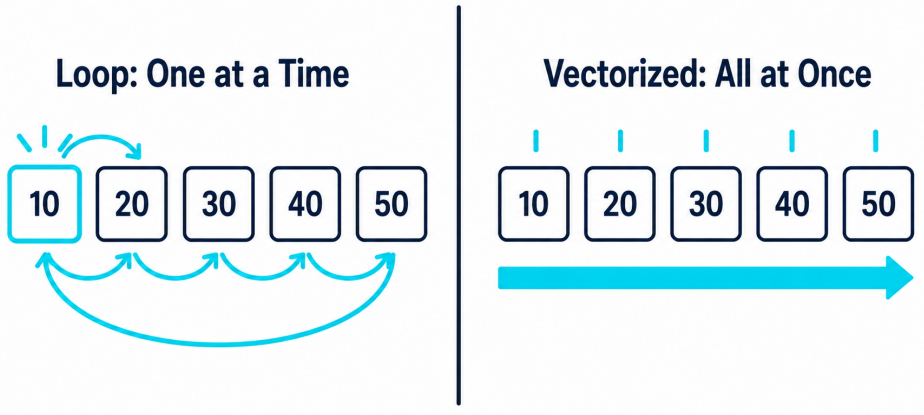
With a NumPy array, you can apply the same operation to every value at once, with no loop at all:

```
readings = np.array([20, 22, 19, 25])
fahrenheit = (readings * 9/5) + 32
```



One operation applied to every value in the array simultaneously.

This single line multiplies every value in the array by 9/5, adds 32 to every result, and returns a new array — all in one operation. This is called vectorization, and it's the central idea behind why NumPy exists: instead of telling the computer to repeat an operation one value at a time, you describe the operation once and apply it to the whole array simultaneously.



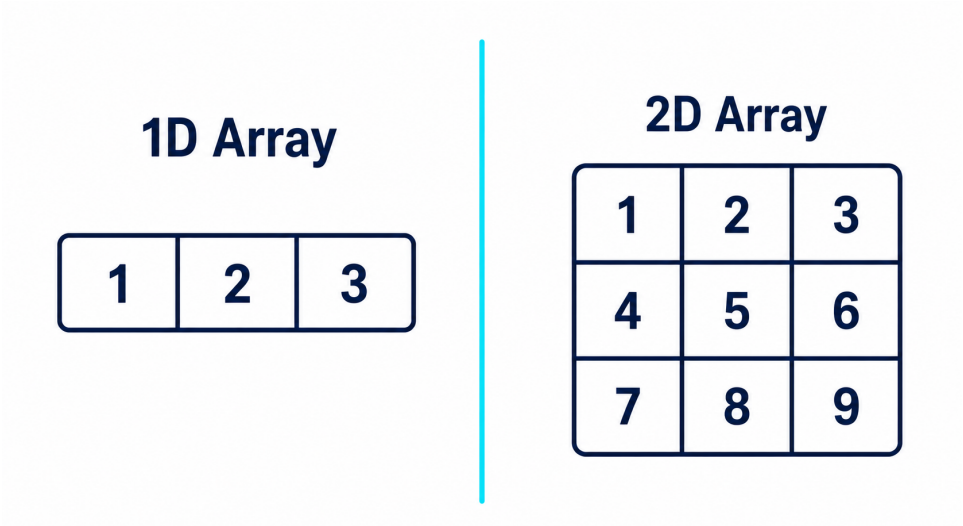
Looping through values one at a time, versus processing the whole array at once.

It's both far less code to write and dramatically faster to run, especially as your data grows into the thousands or millions of values you'll encounter in real machine learning work.

Arrays can have more than one dimension

A regular list is one-dimensional — a single row of values. Real-world data is often two-dimensional or higher — think of a spreadsheet with rows and columns, or a grid of sensor readings taken across multiple sensors over multiple time points. NumPy handles this naturally:

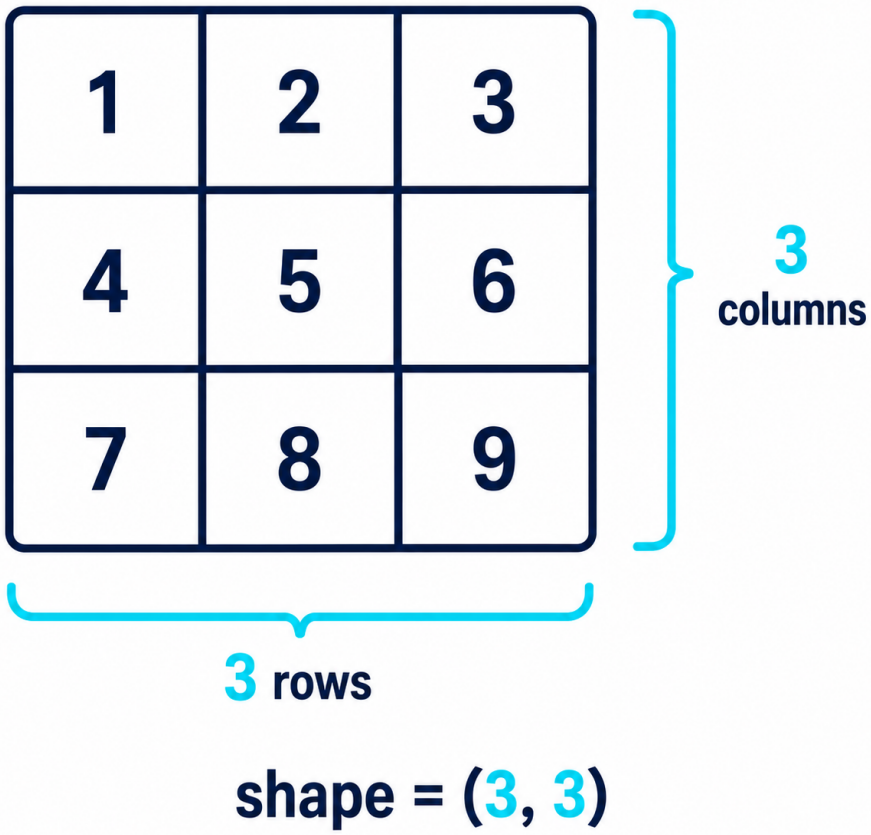
```
sensor_grid = np.array([
    [42, 45, 41],
    [50, 47, 44],
    [39, 41, 43]
])
```



A one-dimensional array versus a two-dimensional array.

This is a two-dimensional array — three rows, three columns. You can check its shape directly:

```
print(sensor_grid.shape)
```



An array's shape describes how many rows and columns it contains.

This would print (3, 3) — three rows, three columns. Understanding "shape" matters a great deal in AI work, because a huge number of errors you'll encounter later come down to arrays not having the shape a function expected. Getting comfortable checking and reasoning about array shape now will save you real frustration later.

Basic operations you'll use constantly

A few NumPy operations come up so often they're worth knowing cold:

```
data = np.array([10, 20, 30, 40, 50])

data.mean()      # average of all values
data.max()       # largest value
data.min()       # smallest value
data.sum()       # total of all values
```

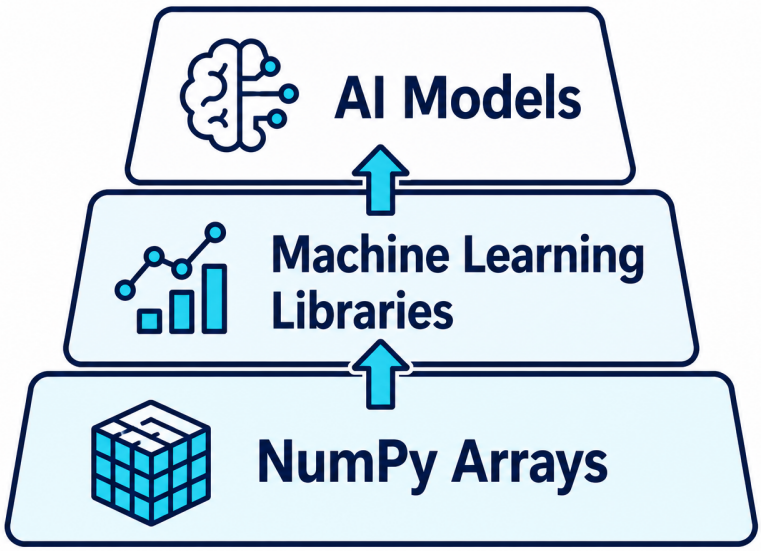


Four operations that run instantly across an entire array.

These run instantly across the entire array, no matter how large it is, without you writing a single loop.

Why this matters for AI specifically

Every machine learning model you'll eventually build operates on numerical data organized into arrays — images are arrays of pixel brightness values, sensor logs are arrays of readings over time, and the internal calculations inside a neural network are, fundamentally, large arrays being multiplied and combined with each other. NumPy is the foundation that nearly every other AI and machine learning library in Python is built on top of.

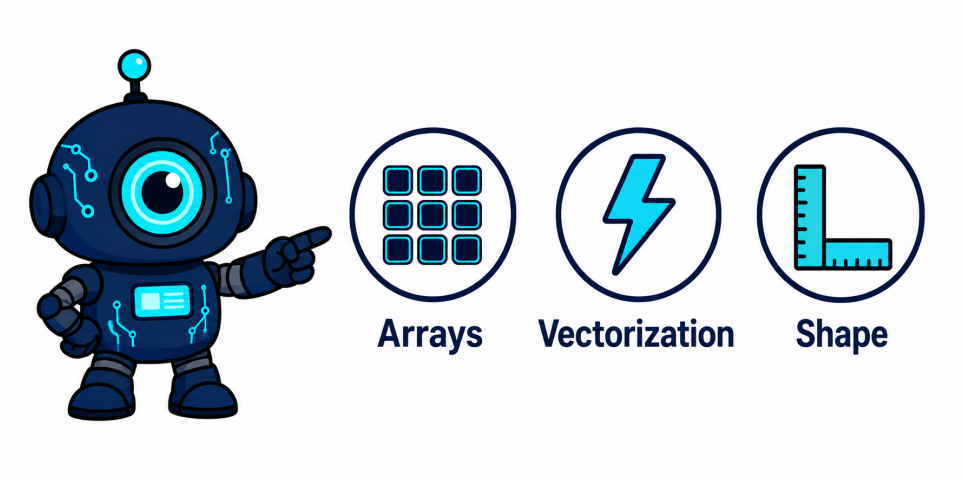


NumPy arrays are the foundation nearly every AI and machine learning library is built on.

You won't always be calling NumPy directly once you get to more advanced tools, but understanding arrays, shape, and vectorized operations now is what makes those advanced tools comprehensible instead of mysterious.

Where this leaves you

By the end of this lesson, you should understand what a NumPy array is and why it exists, be able to create one from a regular list, apply a mathematical operation to an entire array at once without writing a loop, and check an array's shape. The next lesson, Pandas, builds directly on these same ideas, applied to labeled, table-like data.



The three core ideas from this lesson: arrays, vectorization, and shape.